PROJECT REPORT

ON

DJANGO AUTOML STUDIO

A Data Cleaning, Visualization, and Machine Learning Web Application

Submitted by

[Ishika]

[24BCA10124]

Submitted to

[Mr. Ravi Raj]

[Professor]

Academic Session: [2024–2027]

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project supervisor, Mr. Ravi Raj, and the faculty members for their guidance and support during the planning and development of this project. I also thank Chandigarh University for providing the academic environment and resources that helped me understand data mining, web application development, data preprocessing, data visualization, and machine learning. Finally, I acknowledge the documentation and open-source Python libraries that made the implementation of Django AutoML Studio possible.

ABSTRACT

Django AutoML Studio is a local web-based application designed to simplify common stages of a data mining and machine learning workflow. It allows a user to upload a dataset in CSV or Excel format, select a target column, prepare mixed numerical and categorical data, explore visual summaries, train a machine learning model, review evaluation metrics, and request a prediction for a new record. The application is built with Python and Django, while Pandas and NumPy support data handling and Scikit-learn provides preprocessing pipelines and machine learning algorithms.

The preprocessing workflow removes duplicate rows and completely empty columns, removes records with missing target values, caps numerical outliers using the interquartile range (IQR) rule, imputes missing numerical values with the median, imputes missing categorical values with the most frequent value, scales numerical features using StandardScaler, and converts categorical features using OneHotEncoder. The model layer uses RandomForestClassifier for classification and RandomForestRegressor for regression. The application divides eligible data into training and testing sets using an 80:20 split and reports suitable metrics for the selected task.

The dashboard presents a dataset preview, data-quality information, exploratory plots, feature-importance information, and model evaluation results. A prediction form is generated from the retained input features and uses the saved preprocessing-and-model pipeline. The project is intended as an educational local prototype. Performance values depend on the dataset and must be captured from an actual run; this report does not claim an unverified accuracy score.

Project scope note: The application is designed for local use on the user's computer. A real dataset run and final screenshots should be added before formal submission.

TABLE OF CONTENTS

Section	Page
ACKNOWLEDGEMENT	1
ABSTRACT	1
TABLE OF CONTENTS	2
1. INTRODUCTION	3
2. PROBLEM STATEMENT	3
3. OBJECTIVES	3
4. SCOPE OF THE PROJECT	4
5. EXISTING SYSTEM AND PROPOSED SYSTEM	4
6. REQUIREMENTS ANALYSIS	5
7. SYSTEM DESIGN AND ARCHITECTURE	6
8. MAIN MODULES AND WORKFLOW	7
9. METHODOLOGY AND DATA PREPROCESSING	8
10. OUTLIER TREATMENT AND FEATURE TRANSFORMATION	9
11. MACHINE LEARNING ALGORITHM	10
12. MODEL TRAINING, EVALUATION AND PREDICTION	11
13. IMPLEMENTATION DETAILS	12
14. TESTING AND VALIDATION	13
15. RESULTS AND DISCUSSION	14
16. LIMITATIONS AND DISADVANTAGES	15
17. FUTURE ENHANCEMENTS	16
18. CONCLUSION	16
REFERENCES	17
APPENDIX A: USER GUIDE	17
APPENDIX B: SUBMISSION CHECKLIST	17

Page numbering starts after the cover page. The final page numbers may be adjusted if institution-specific additions are made.

1. INTRODUCTION

Data is generated continuously by businesses, institutions, healthcare organisations, educational platforms, and online services. A dataset may contain hundreds or thousands of records with numerical and categorical fields. Before such data can be used for machine learning, it must be inspected, cleaned, transformed, and analysed. Performing each task manually in separate tools can be slow and difficult for a beginner to repeat.

Django AutoML Studio combines these activities in a single local web application. A user uploads a dataset, chooses the value to be predicted, and runs the analysis. The application prepares the input features, creates exploratory visualizations, trains a baseline Random Forest model, displays evaluation results, and provides a form for predictions on new observations. It supports both classification, where the output is a category, and regression, where the output is a numerical value.

2. PROBLEM STATEMENT

Raw datasets often contain missing values, duplicate records, empty columns, unusually large or small numerical values, and text-based categories. These issues can prevent a machine learning algorithm from working correctly or can reduce the quality of its predictions. Users may also find it difficult to understand the dataset through tables alone, prepare categorical features, train a model, and interpret the model evaluation results.

The project addresses this problem by providing a guided interface for dataset upload, data cleaning, feature transformation, exploratory data analysis, model training, evaluation, and single-record prediction. The application is intended to reduce repetitive coding for a basic machine learning experiment; it does not replace expert data validation or domain knowledge.

3. OBJECTIVES

- Accept CSV, XLSX, and XLS datasets through a web upload interface.
- Display dataset dimensions, column information, missing values, duplicates, and a small preview.
- Handle empty columns, duplicate records, missing target values, missing feature values, and numerical outliers.
- Encode categorical features and scale numerical features for model training.
- Generate exploratory plots and feature-importance information.
- Support classification and regression using Random Forest models.
- Evaluate the model on a held-out test set using appropriate performance metrics.
- Allow the user to enter new feature values and receive a prediction.

4. SCOPE OF THE PROJECT

Django AutoML Studio is designed for use on a desktop or laptop through a modern web browser. It accepts local CSV and Excel files up to 25 MB. The project is suitable for datasets containing numerical and categorical attributes and a target column that can be used for supervised learning. The application can perform a classification task or a regression task. It can be demonstrated with the Kaggle Titanic training dataset, which contains 891 records and mixed feature types; in that example, the Survived column can be selected as the classification target.

The application is designed to provide a baseline workflow rather than an enterprise AutoML platform. The user is responsible for selecting a meaningful target column, checking whether the chosen task is suitable, reviewing the dataset quality, and interpreting the results. The current implementation is local-first and does not provide public cloud hosting, user accounts, or automated model comparison.

5. EXISTING SYSTEM AND PROPOSED SYSTEM

5.1 Existing System

In a conventional manual process, a user may inspect data in a spreadsheet, clean it with separate scripts, create charts in another tool, write machine learning code in a notebook, and build a separate interface for prediction. These steps require knowledge of several tools and can be difficult to repeat consistently.

5.2 Proposed System

The proposed system brings dataset upload, preprocessing, visualization, model training, model evaluation, and prediction into one Django-based application. Python libraries perform the data and machine learning operations, while Django manages page requests, uploaded files, and session-based results. The trained pipeline is saved locally so it can be used for predictions without retraining for every input.

5.3 Advantages

- One interface for data preparation, visualization, model evaluation, and prediction.
- Reduces the need to write a separate preprocessing script for every small experiment.
- Works with different suitable CSV and Excel datasets.
- Handles numerical and categorical input features in a common preprocessing pipeline.
- Uses a saved pipeline to keep prediction-time preprocessing consistent with training.

6. REQUIREMENTS ANALYSIS

6.1 Functional Requirements

- Upload a CSV, XLSX, or XLS dataset and validate the file type and file size.
- Read the dataset and display the number of rows, columns, missing values, duplicates, and preview records.
- Allow the user to select a target column and choose Auto Detect, Classification, or Regression.
- Clean duplicate records, remove completely empty columns, and remove rows with missing target values.
- Apply IQR outlier capping, median/mode imputation, numerical scaling, and categorical encoding.
- Generate EDA charts and feature-importance information.
- Train a Random Forest model and display task-specific evaluation metrics.
- Generate a custom prediction form from the model input features and return a prediction.

6.2 Non-Functional Requirements

- Usability: the dashboard should present the workflow in a clear, understandable sequence.
- Compatibility: a modern Windows environment and browser are expected for the supplied launcher.
- Performance: processing time depends on dataset size, number of columns, data types, CPU, and available memory.
- Local data handling: uploaded files and trained models are stored on the local computer.
- Reliability: users should verify the selected target, task type, input fields, and model metrics before relying on predictions.
- Security: the supplied application is intended for localhost use and is not configured as a public multi-user service.

6.3 Hardware and Software Requirements

Category	Requirement
Hardware	Desktop or laptop; adequate RAM and CPU for the selected dataset; free disk space for uploads and saved models.
Operating system	Windows 10/11 recommended for the supplied batch launch and EXE build scripts.
Runtime	Python 3.11 or 3.12 for source-based setup; a compatible Windows build environment for creating the EXE.
Framework	Django 5.1 series; Waitress for the local WSGI server.
Libraries	Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn, Joblib, openpyxl, xlrd.
Browser	A current browser such as Chrome, Edge, or Firefox.

7. SYSTEM DESIGN AND ARCHITECTURE

7.1 Architecture Overview

The application follows a local web architecture. The user interacts with the dashboard in a browser. Django receives upload, analysis, prediction, and reset requests through URL routes and view functions. The analyzer engine reads the dataset with Pandas, prepares features with Scikit-learn, creates plots with Matplotlib and Seaborn, and stores fitted model pipelines with Joblib. Session information is stored in a local SQLite database. The local server listens only on 127.0.0.1 at port 8765.

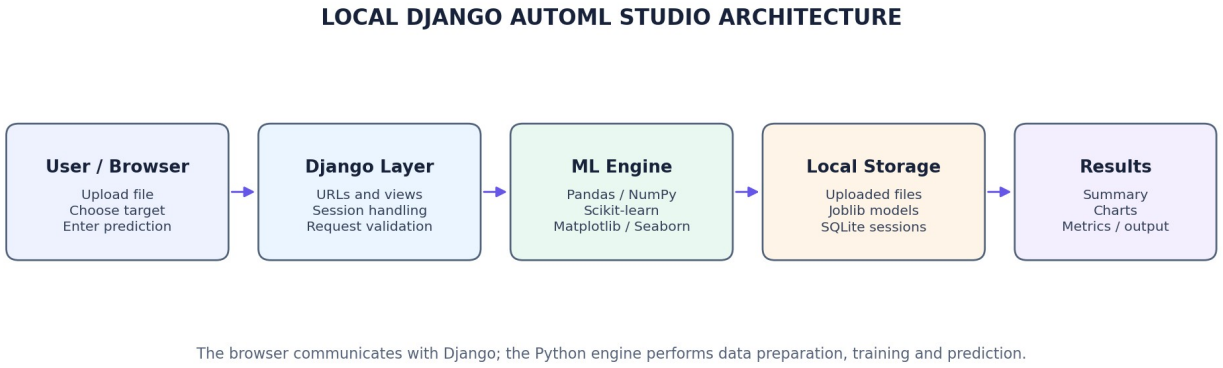


Figure 1: Local architecture of Django AutoML Studio

The application has three main technical layers: the presentation layer (HTML/CSS dashboard), the Django application layer (URLs, views, validation, and session handling), and the analysis layer (data reading, preprocessing, visualization, training, evaluation, and prediction). Local uploads and model files are stored separately from the application executable so that they can persist between launches.

7.2 Data and Model Storage

On Windows, application data is stored under the user's LOCALAPPDATA folder in a DjangoAutoMLStudio directory. The directory contains upload files, trained Joblib model files, a local SQLite session database, and Matplotlib configuration data. The EXE or source folder therefore does not need to contain the user's working dataset.

8. MAIN MODULES AND WORKFLOW

8.1 Main Modules

Module	Purpose	Main output
Dataset Upload	Accept and read CSV or Excel files; validate extension and size.	Dataset columns and rows.
Data Summary	Calculate dimensions, duplicates, missing values, and preview rows.	Dataset summary and preview.
Preprocessing	Remove duplicates and empty columns; impute, cap, encode, and scale features.	Prepared feature matrix.
EDA and Visualization	Generate target, distribution, box, scatter, and correlation plots.	Base64-encoded chart images.
Feature Screening	Exclude selected ID-like/high-cardinality text fields and report forest importance.	Retained feature list and importance values.
Model Training	Train Random Forest classifier or regressor.	Fitted ML pipeline.
Evaluation	Calculate held-out test metrics and a confusion matrix or actual-vs-predicted plot.	Model performance summary.
Prediction	Create a form from retained inputs and score one new record.	Predicted value and classification confidence, when applicable.

8.2 Overall Workflow

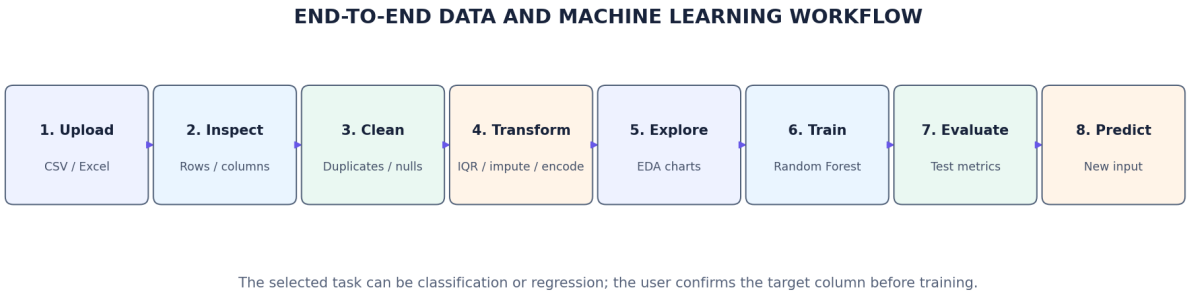


Figure 2: End-to-end processing and prediction workflow

The user first uploads a dataset and selects the target column. The application then cleans and transforms the eligible input features, generates EDA charts, splits the records into training and test subsets, trains the selected Random Forest model, calculates evaluation metrics, refits the pipeline on all eligible records, saves it, and enables prediction for a new record.

9. METHODOLOGY AND DATA PREPROCESSING

9.1 Dataset Loading and Inspection

The read_dataset() function reads CSV files with Pandas and uses a Latin-1 fallback if a Unicode decoding error occurs. Excel files with .xlsx or .xls extensions are read through Pandas Excel support. The application checks that the uploaded dataset is not empty, trims column labels, creates a name for blank headers, and makes duplicate column names unique. The upload view accepts files up to 25 MB.

For an assignment demonstration, the project README recommends the Kaggle Titanic training file (train.csv). It contains 891 training records and both numerical and categorical attributes. Survived can be selected as a binary target, while features such as passenger class, age, sex, fare, and family counts can be used as inputs. The dataset is downloaded by the user from Kaggle; the application does not download it automatically.

9.2 Target Selection and Task Detection

The target is the column that the model is expected to predict. The user selects this column from the available dataset fields. The application supports Auto Detect, Classification, and Regression. In Auto Detect mode, a non-numeric target or a numeric target with 20 or fewer distinct values is treated as classification; a numeric target with more than 20 distinct values is treated as regression. This is a heuristic, so the user should confirm or manually override the choice.

Classification requires at least two target classes. Regression converts the target to numeric values. At least 30 rows with usable target values are required to train; the application separately displays a warning if the cleaned dataset contains fewer than 500 rows, because the assignment asks for 500 or more records.

9.3 Data Cleaning and Missing Values

- Duplicate removal: completely duplicated records are removed before model training.
- Empty columns: columns containing no usable values are removed.
- Missing target: rows with a missing target value are removed because the model cannot learn a target label/value from them.
- Numeric feature gaps: SimpleImputer(strategy="median") replaces missing values with the median learned from the training data.
- Categorical feature gaps: SimpleImputer(strategy="most_frequent") replaces missing values with the most common category learned from training data.

The preprocessing operations for model fitting are placed inside a Scikit-learn Pipeline and ColumnTransformer. This helps ensure the imputation and transformation steps used for prediction are the same as those learned during training.

10. OUTLIER TREATMENT AND FEATURE TRANSFORMATION

10.1 Outlier Detection Using IQR

An outlier is a value that is unusually far from the central range of other values. The project uses the interquartile range (IQR) rule for numerical features. IQR is calculated as Q3 minus Q1, where Q1 is the 25th percentile and Q3 is the 75th percentile.

$$IQR = Q3 - Q1$$

$$\text{Lower limit} = Q1 - 1.5 \times IQR$$

$$\text{Upper limit} = Q3 + 1.5 \times IQR$$

Values outside the calculated limits are capped to the nearest boundary rather than deleting the entire record. The custom IQRCapper transformer learns its limits from the training subset and applies those learned limits to later data, reducing the risk of using test-set information during model fitting. The EDA summary also reports counts of values outside the IQR limits.

10.2 Numerical Feature Transformation

Numerical features pass through the IQRCapper, median imputation, and StandardScaler steps. StandardScaler transforms a numerical value using z = (x - mean) / standard deviation. The scaling parameters are learned from the training data and reused for test data and future predictions.

10.3 Categorical Feature Transformation

Categorical columns are imputed using their most frequent value and converted into indicator columns using OneHotEncoder. The encoder is configured to ignore unknown categories at prediction time and limit the number of categories per feature to 40. This allows the pipeline to accept a category that was not present in the training sample without failing on an unknown encoded value.

10.4 Feature Screening and Importance

Before model fitting, the application checks for identifier-like fields, including common names such as ID, UUID, CustomerID, UserID, and similar record identifiers. It also excludes high-cardinality text columns when they contain more than 60 distinct values and a high unique-value ratio. If this screening would remove every feature, the application falls back to the original candidate feature list. After training, Random Forest feature_importances_ are used to display up to 12 influential transformed features. This is a model-specific importance ranking, not proof of causation and not a full automatic correlation-based feature elimination process.

10.5 Exploratory Data Analysis

EDA charts include a target distribution plot, box plots and histograms for up to three numerical features, a scatter plot for the first two eligible numerical features (using a sample of up to 700 rows), and a correlation heatmap for up to 12 numerical columns. A confusion matrix is added for classification, while an actual-versus-predicted plot is added for regression.

11. MACHINE LEARNING ALGORITHM

11.1 Supervised Learning

The project uses supervised machine learning. In supervised learning, the training dataset contains input features and a known target value. The algorithm learns relationships between the features and the target, then uses the learned pattern to estimate the target for new records. Classification predicts a category, while regression predicts a numerical value.

11.2 Random Forest

Random Forest is an ensemble learning algorithm that combines the outputs of many decision trees. Each tree is trained using a sample of the training data and considers subsets of features when building splits. For classification, the trees vote for a class and the forest returns the class with the strongest vote. For regression, the tree predictions are averaged to produce a numerical estimate.

The project uses RandomForestClassifier when the selected task is classification and RandomForestRegressor when the task is regression. Both models are configured with 180 trees, a fixed random_state of 42, and parallel processing using all available CPU cores. The classifier additionally uses class_weight="balanced_subsample" to help account for unequal class frequencies in the bootstrap samples.

11.3 Why Random Forest Is Used

- It can model non-linear relationships between input features and the target.
- It can be used for both classification and regression tasks.
- Combining many trees often provides more stable predictions than relying on one decision tree.
- It supports a feature-importance output that can help users inspect which transformed features influenced the fitted model most.

11.4 Simplified Algorithm Steps

- Prepare numerical and categorical input features and the selected target.
- Split the eligible records into training and testing subsets.
- Fit the preprocessing pipeline and Random Forest model on the training subset.
- Generate predictions for the held-out test subset.
- Calculate classification or regression evaluation metrics.
- Refit the complete pipeline using all eligible records and save it for future prediction.

The current implementation uses Random Forest as its baseline model. It does not automatically compare several different algorithms or tune model hyperparameters.

12. MODEL TRAINING, EVALUATION AND PREDICTION

12.1 Training and Testing

The application uses train_test_split with test_size=0.2 and random_state=42. This creates an 80% training subset and a 20% test subset. For classification, stratified splitting is used when every class has at least two rows; if stratification cannot be applied, the code falls back to a regular random split. The test subset is kept separate during the first model fit so that the model can be evaluated on records not used in fitting.

12.2 Classification Metrics

Metric	Meaning
Accuracy	Proportion of test records for which the predicted class matches the actual class.
Precision	Of the records predicted as a class, the proportion that truly belong to that class.
Recall	Of the records that truly belong to a class, the proportion correctly identified by the model.
F1-score	Harmonic mean of precision and recall; useful when both false positives and false negatives matter.
Confusion matrix	Table showing actual versus predicted classes, including correct and incorrect predictions.

The application calculates weighted precision, weighted recall, and weighted F1-score for classification, along with accuracy and a confusion matrix. Weighted averaging accounts for the number of test records in each class.

12.3 Regression Metrics

Metric	Meaning
MAE	Mean absolute difference between actual and predicted numerical values.
RMSE	Square root of mean squared error; larger errors receive a stronger penalty.
R^2 score	Measures the proportion of target-value variation explained by the model relative to a baseline.

12.4 Saving and Using the Model

After the test metrics are calculated, the application fits the pipeline again on all eligible rows. It saves the pipeline, retained feature names, target name, and task type to a Joblib file. The prediction form is generated from the retained features: numerical fields are pre-filled with a median suggestion and categorical fields use a mode suggestion with common category choices. When the user submits the form, the saved pipeline applies the same preprocessing and returns a predicted class or value. For classification, the interface also reports the largest predicted class probability as a confidence estimate.

13. IMPLEMENTATION DETAILS

13.1 Technology Stack

Technology / Library	Role in the project
Python	Main programming language for the backend and ML engine.
Django	Handles routes, views, file-upload requests, sessions, and HTML rendering.
HTML and CSS	Provide the webpage structure, dashboard layout, forms, cards, and visual theme.
JavaScript	Supports client-side interface interactions.
Pandas and NumPy	Read, inspect, and manipulate tabular and numerical data.
Scikit-learn	Provides IQR transformer integration, imputers, encoding, scaling, pipelines, Random Forest, splitting, and metrics.
Matplotlib and Seaborn	Create exploratory plots and model evaluation charts.
Joblib	Save and reload fitted preprocessing and model pipelines.
SQLite	Stores local Django session data.
Waitress and PyInstaller	Serve the local Django app and package a Windows executable, respectively.

13.2 Dashboard Features

- Upload panel for supported CSV and Excel files.
- Dataset summary including row and column counts, missing-value information, duplicate count, and an eight-row preview.
- Target-column and task-type selection before training.
- Visual analytics area with EDA plots and model evaluation graphics.
- Model metrics and a feature-importance list.
- Prediction form generated from the features retained during model training.
- Start Over action to clear the current session state.

13.3 Project Folder Structure

Path	Purpose
automl_studio/	Django settings, URL configuration, and WSGI entry point.
analyzer/	Django views and the data preprocessing, EDA, training, and prediction engine.
templates/analyzer/index.html	Main dashboard interface.
run_app.py	Starts the local Waitress server and opens the browser.
run_once_setup.bat	Creates a virtual environment, installs dependencies, and launches the app.
build_windows_exe.bat	Builds the Windows executable using PyInstaller.
requirements.txt	Lists the required Python packages.

14. TESTING AND VALIDATION

Testing checks whether each major operation responds as expected. The table below lists suggested test cases based on the supplied source code. The actual-result column is intentionally left as pending because the final Windows application run has not yet been verified for this report. The entries should be updated after executing each test on the target computer.

Test case	Input / action	Expected result	Actual result
Valid CSV upload	Upload a valid CSV file.	Dataset is read and columns are available.	Pending local test
Valid Excel upload	Upload .xlsx or .xls.	First worksheet is read.	Pending local test
Unsupported file	Upload a non-CSV/Excel file.	Clear validation message is shown.	Pending local test
File-size limit	Upload a file over 25 MB.	Upload is rejected with a size message.	Pending local test
Missing feature values	Use a dataset with blank input cells.	Median/mode imputation is applied in the model pipeline.	Pending local test
Duplicate records	Include repeated rows.	Duplicate rows are removed.	Pending local test
Empty target	Select a target column with no usable values.	Analysis is rejected with an error.	Pending local test
Small dataset	Use fewer than 30 eligible target rows.	Training is rejected with a validation error.	Pending local test
Classification	Choose a categorical or low-cardinality target.	Classifier metrics and confusion matrix are generated.	Pending local test
Regression	Choose a continuous numerical target.	MAE, RMSE, R^2 and actual-vs-predicted plot are generated.	Pending local test
Single prediction	Enter values for the required input fields.	A predicted class/value is displayed.	Pending local test

Do not mark a test as passed until it has actually been run. Keep screenshots or short logs as evidence for important test cases.

15. RESULTS AND DISCUSSION

The application is designed to produce results from the dataset selected by the user. Since model performance depends on the file, target variable, class balance, missing values, and train-test split, no fixed accuracy score is appropriate for every dataset. The recommended Titanic example has 891 training rows; after uploading it, the user should select Survived and run the classification workflow. The application will then report the actual cleaned row count, train/test sizes, and performance metrics.

15.1 Results Table to Complete

Measure	Value from actual run
Dataset filename and source	To be recorded after execution
Rows and columns uploaded	To be recorded after execution
Rows after cleaning	To be recorded after execution
Duplicate rows removed	To be recorded after execution
Empty columns removed	To be recorded after execution
Training rows / testing rows	To be recorded after execution
Selected target and task	To be recorded after execution
Accuracy / Precision / Recall / F1 (classification)	To be recorded after execution
MAE / RMSE / R^2 (regression)	To be recorded after execution
Most important transformed feature	To be recorded after execution

15.2 Screenshots to Insert

- Figure 3: Dashboard with the dataset uploaded and target column selected.
- Figure 4: Dataset preview and preprocessing summary.
- Figure 5: Exploratory data analysis charts and feature importance.
- Figure 6: Model evaluation metrics and confusion matrix or regression plot.
- Figure 7: Prediction form and an actual prediction result.

The screenshots must be captured from the working application. The architecture and workflow figures in this report are explanatory diagrams, not evidence of a completed runtime test.

16. LIMITATIONS AND DISADVANTAGES

16.1 Limitations of the Current Implementation

- The application currently uses Random Forest as its implemented baseline model; it does not compare multiple algorithms automatically.
- Automatic task detection is heuristic. A numeric target with 20 or fewer unique values may be treated as classification even when the problem is intended as regression; users should confirm the task.
- The application requires at least 30 rows with a usable target for training. It warns when the cleaned dataset has fewer than 500 rows, but it does not enforce the assignment minimum as a hard stop.
- The upload limit is 25 MB and the current supported formats are CSV, XLSX, and XLS.
- The implementation uses one 80:20 hold-out split rather than k-fold cross-validation.
- Hyperparameter tuning and automated comparison of candidate models are not implemented.
- Identifier-like and high-cardinality text fields are excluded by heuristic rules; these rules may remove a field that is useful for a particular domain.
- Feature importance is a model-specific indicator and does not establish causation.
- The local application does not include user authentication, role-based access, cloud deployment, or multi-user access controls.
- Uploaded files and saved model files remain in the local application-data folder until manually removed; Start Over clears session references but does not delete every saved file.

16.2 General Disadvantages

- Random Forest can require more memory and training time than a simple linear model, especially as the dataset and number of trees grow.
- The model is less directly interpretable than a small decision tree or linear model.
- Predictions can be unreliable if the uploaded data is unrepresentative, biased, incomplete, or contains target leakage.
- A high evaluation score on one split does not guarantee similar performance on future data.

17. FUTURE ENHANCEMENTS

The project can be extended in several ways to make it more flexible, reliable, and suitable for larger data analysis tasks.

- Add more algorithms such as Logistic Regression, Decision Tree, K-Nearest Neighbors, Support Vector Machine, Gradient Boosting, and XGBoost, where compatible.
- Add cross-validation and a model-comparison table to evaluate more than one candidate model.
- Add hyperparameter tuning to search for suitable model settings instead of using only fixed parameters.
- Add stronger data validation, configurable treatment of missing values, and optional transformations.
- Add more feature-selection methods such as correlation analysis, mutual information, and model-based selection thresholds.
- Add ROC-AUC, PR-AUC, calibration, and threshold-selection tools for classification.
- Add explainability features to describe individual predictions and the contribution of input attributes.
- Add downloadable PDF or Excel reports containing data-quality summaries, visualizations, and model metrics.
- Add user login, access controls, secure storage, model versioning, and controlled retraining before any multi-user deployment.
- Improve support for large datasets through memory-aware loading, progress indicators, and background task processing.

18. CONCLUSION

Django AutoML Studio demonstrates how a data mining and machine learning workflow can be brought together in a local web application. It provides dataset upload, data inspection, duplicate and missing-value handling, IQR-based outlier capping, numerical scaling, categorical encoding, exploratory visualization, Random Forest model training, model evaluation, and custom prediction. Django connects the dashboard to a Python analysis engine, while Scikit-learn provides reusable preprocessing and model pipelines.

The project is useful as an educational prototype because it reduces repetitive setup work and makes key machine learning stages visible through a web interface. Its outputs must still be checked carefully: the selected target and task must be meaningful, the dataset must be representative, and evaluation values must be taken from a real run. Future work can add model comparison, cross-validation, tuning, stronger validation, and secure multi-user deployment.

REFERENCES

1. Kaggle. Titanic - Machine Learning from Disaster. https://www.kaggle.com/competitions/titanic/data

2. Django Software Foundation. Django documentation. https://docs.djangoproject.com/

3. pandas development team. pandas documentation. https://pandas.pydata.org/docs/

4. NumPy developers. NumPy documentation. https://numpy.org/doc/

5. scikit-learn developers. Random forests. https://scikit-learn.org/stable/modules/ensemble.html#forest

6. scikit-learn developers. Pipeline and composite estimators. https://scikit-learn.org/stable/modules/compose.html

7. scikit-learn developers. Imputation of missing values. https://scikit-learn.org/stable/modules/impute.html

8. scikit-learn developers. Preprocessing data. https://scikit-learn.org/stable/modules/preprocessing.html

9. scikit-learn developers. Model evaluation. https://scikit-learn.org/stable/modules/model_evaluation.html

10. Matplotlib development team. Matplotlib documentation. https://matplotlib.org/stable/

11. Seaborn development team. Seaborn documentation. https://seaborn.pydata.org/

12. Joblib developers. Joblib documentation. https://joblib.readthedocs.io/

13. Waitress documentation. https://docs.pylonsproject.org/projects/waitress/en/stable/

14. PyInstaller documentation. https://pyinstaller.org/en/stable/

APPENDIX A: USER GUIDE

- Install Python 3.11 or 3.12 on Windows and select the option to add Python to PATH.
- Extract the project ZIP into a normal writable folder.
- Double-click run_once_setup.bat. On first launch, the script creates a local virtual environment and installs the listed packages.
- When the app opens in a browser, upload a CSV, XLSX, or XLS file.
- Select the target column and confirm whether the task is classification or regression.
- Run analysis and review the dataset summary, charts, feature importance, and model metrics.
- Complete the generated prediction form and submit the input values to obtain a prediction.
- Use Start Over to clear the current session and begin another dataset workflow.

APPENDIX B: SUBMISSION CHECKLIST

- Run the project successfully on the target Windows computer.
- Use a real Kaggle dataset with at least 500 records and mixed numerical/categorical fields.
- Fill in the results table using observed values from the actual application run.
- Insert genuine screenshots for upload, preprocessing, dashboard, evaluation, and prediction.
- Complete the testing table with actual pass/fail outcomes and retain evidence.
- Check the final report against the university requirements for declaration, formatting, binding, and references.

Accuracy and other performance values have deliberately not been fabricated. Complete the results and screenshot sections after a successful run of the application.